

2026年春

程序设计实习：C++程序设计

第十讲 C++进阶内容

贾川民
北京大学



- 课程安排：4月15日周三， STL/C++进阶内容
- 课程安排：4月17日周五， C++新标准， 合班上课， 理教208
- 课程安排：4月22日周三， Qt技术交流， 合班上课， 理教208
- 期中考试：4月29日周三， 上午10:00-12:00， 考场安排待定
- 关于上机：
 - 本周， 正常上机练习；
 - 下周， 模拟考试（去年真题）

大作业概览

- 大作业实现一个带界面的综合性项目，组队进行，提交报告/代码/演示视频，优秀项目进行路演
- 两种语言：使用C++或Python来完成，二选一
- 两个赛道：生活工具/游戏；元学习 (学习程设的工具/游戏)
二选一

请选择您的语言：

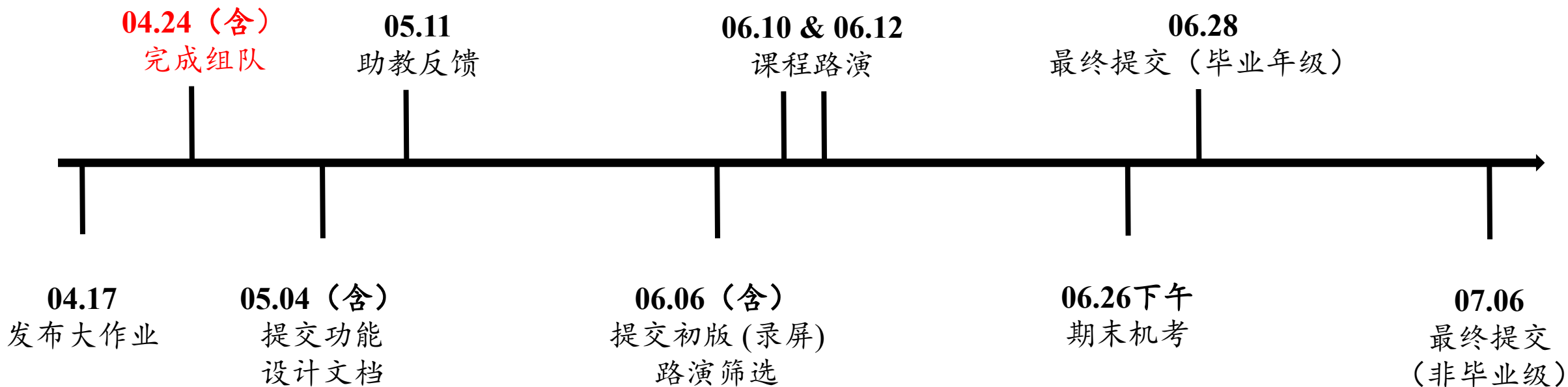


二选一



大作业安排时间线

(供参考, 可能有调整)



大作业安排

- 组队事宜：

- 04月24日前完成组队。
- 3人一队（可以跨班组队），确定队名和队长，由**组长**填写（<https://v.wjx.cn/vm/rWUcsmW.aspx#>），组员请勿重复填写
- 如果有同学不打算自己组队，想等助教分配，**也需要填写问卷**，并在队名里标注“希望助教帮忙分配”
- 确认分组后，每支队伍会分配负责助教，具体对接项目立项检查、环节沟通

- 在C语言中，类型转换有几种方式：

(expression) 在表达式外边加括号，由编译器来决定怎么改变

new_type (expression) 强制类型括号住表达式

(new_type) expression 括号住强制类型

- 若没有了解每个转换的细节，就有可能出现问题，例如：

指针指向不应该指向的区域：

出现野指针或者指向位置错误

计算数值被截去

□ 为了C语言类型转换中语义模糊和固有的危险陷阱,

☒ C语言不去判断所要操作的类型转换是否合理

□ C++的四种 "cast" 运算符

- **static_cast**
- **dynamic_cast**
- **reinterpret_cast**
- **const_cast**

□ C++强制类型转换运算符

强制类型转换运算符<要转换到的类型> (待转换表达式)

1. static_cast

□ static_cast 用于进行比较 "自然" 和低风险的转换

e.g. 整型和实数型, 字符型之间互相转换

□ static_cast 不能用于不同类型的指针之间互相转换

不能用于整型和指针之间的互相转换

不能用于不同类型的引用之间的转换



//program 20.1.1.cpp static_cast示例

```
#include <iostream>
```

```
using namespace std;
```

```
class A{
```

```
    public:
```

```
        operator int() { return 1; }
```

```
        operator char * () { return NULL; }
```

```
};
```

```
int main(){
```

```
    A a;
```

```
    int n;
```

```
    char * p = "New Dragon Inn";
```

```
    n = static_cast<int>(3.14); // n 的值变为 3
```

```
    n = static_cast<int>(a);    //调用a.operator int, n的值变为 1
```

```
p = static_cast<char*>(a);  
//调用a.operator char *, p的值变为 NULL  
n = static_cast<int> (p);  
//编译错误, static_cast不能将指针转换成整型  
p = static_cast<char*>(n);  
//编译错误, static_cast不能将整型转换成指针  
return 0;  
}
```

2. reinterpret_cast

- 不同类型的指针之间的转换
- 不同类型的引用之间转换
- 指针和能容纳得下指针的整数类型之间的转换
转换的时候, 执行的是**逐个比特拷贝**的操作

//program 20.1.2.cpp reinterpret_cast 示例

```
#include <iostream>
using namespace std;
class A{
    public:
        int i;
        int j;
        A(int n):i(n), j(n) { }
};
int main(){
    A a(100);
    int & r = reinterpret_cast<int&>(a); //强行让 r 引用 a
    r = 200; //把 a.i 变成了 200
    cout << a.i << “, ” << a.j << endl; // 输出 200, 100
    int n = 300;
```

```
A * pa = reinterpret_cast<A*> (& n); //强行让 pa 指向 n
pa->i = 400; // n 变成 400
pa->j = 500; //此条语句不安全, 很可能导致程序崩溃
cout << n << endl; // 输出 400
long long la = 0x12345678abcdLL;
pa = reinterpret_cast<A*>(la);
// la 太长, 只取低32位0x5678abcd拷贝给pa
unsigned int u = reinterpret_cast<unsigned int>(pa);
//pa逐个比特拷贝到u
cout << hex << u << endl; //输出 5678abcd
typedef void (* PF1) (int); //函数指针, 后面那个括号
typedef int (* PF2) (int, char *); //是函数的参数
PF1 pf1;    PF2 pf2;
pf2 = reinterpret_cast<PF2>(pf1);
//两个不同类型的函数指针之间可以互相转换
}
```

输出结果:
200, 100
400
5678abcd

4. dynamic_cast

- 用于将多态基类的指针或引用，强制转换为派生类的指针或引用
- 且能够检查转换的安全性
- 对于不安全的指针转换，转换结果返回NULL指针
- dynamic_cast不能用于将非多态基类的指针或引用，强制转换为派生类的指针或引用

回顾C++存在基类时的类型转换规则

f(派生类) y(基类): f:对象/对象地址; y:对象/引用/指针
基类指针 (强制指针类型转换)派生类指针

//program 20.1.3.cpp dynamic_cast示例

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Base{ //有虚函数, 因此是多态基类  
    public:
```

```
        virtual ~Base() { }
```

```
};
```

```
class Derived : public Base { };
```

```
int main()
```

```
{
```

```
    Base b;
```

```
    Derived d;
```

```
    Derived * pd;
```

```
    pd = reinterpret_cast<Derived*> (&b);
```



```
if( pd == NULL) //此处pd不会为NULL
    // reinterpret_cast不检查安全性，总是进行转换
    cout << "unsafe reinterpret_cast" << endl; //不会执行
pd = dynamic_cast<Derived*> (&b);
if( pd == NULL) //结果会是NULL，因为 &b不是
    //指向派生类对象，此转换不安全
    cout << "unsafe dynamic_cast1" << endl; //会执行
Base * pb = & d;
pd = dynamic_cast<Derived*> (pb); //安全的转换
if( pd == NULL) //此处pd 不会为NULL
    cout << "unsafe dynamic_cast2" << endl; //不会执行
return 0;
}
```

输出结果:

unsafe dynamic cast1

Derived & r = dynamic_cast<Derived&>(b);

那该如何判断该转换是否安全呢？

答案：不安全则抛出异常

- C++异常处理基础: try, throw, catch
- 异常声明 (exception specification)
- 意外异常 (unexpected exception)
- 异常处理的作用
- 动态内存管理的异常处理

new



- 程序运行中总难免发生错误
 - 数组元素的下标超界, 访问NULL指针
 - 除数为0
 - 动态内存分配new需要的存储空间太大
 - ...



- 引起这些异常情况的原因：
 - 代码质量不高, 存在bug
 - 输入数据不符合要求
 - 程序的算法设计时考虑不周到
 - ...

□ 我们总希望在发生异常情况时

- 不只是简单地终止程序运行

- 能够反馈异常情况的信息：

哪一段代码发生的，什么异常

- 能够对程序运行中已发生的事情做些处理：

取消对输入文件的改动/释放已经申请的系统资源

- ...

□ 通常的做法:

在预计会发生异常的地方, 加入相应的代码,
但这种做法并不总是适用的

```
...           //对文件A进行了相关的操作  
fun( arg, ... ); //可能发生异常  
...
```


□ **caller** 该如何知道 **fun(arg, ...)** 是否发生异常

没有发生异常, 可以继续执行

发生异常, 应该在结束程序运行前还原对文件A的操作

□ **fun(arg, ...)** 是别人已经开发好的代码

fun(arg, ...) 的编写者不知道其他人会如何使用这个函数

fun(arg, ...) 会出现在表达式中, 通过返回值的方式区分是否发生异常

不符合编写程序的习惯

可能发生多种异常, 通过返回值判断也很麻烦

□ throw语句

throw 表达式;

抛出一个异常, 异常是一个表达式

其值类型可以是一个基本类型, 也可以是个类

□ try ...catch语句

```
try{
```

语句组

```
}
```

```
catch(异常类型) {
```

异常处理代码

```
}
```

```
#include <iostream>
using namespace std;
int main() {
    double m , n;
    cin >> m >> n;
    try {
        cout << "before dividing." << endl;
        if( n == 0 )
            throw -1; //抛出int类型异常
        else
            cout << m / n << endl;
        cout << "after dividing." << endl;
    }
```

```
catch(double d) {  
    cout << "catch(double) " << d << endl;  
}  
catch(int e) {  
    cout << "catch(int) " << e << endl;  
}  
cout << "finished" << endl;  
return 0;
```

}

程序运行结果如下:

9 6 

before dividing.

1.5

after dividing.

finished

//program 20.4.2.cpp 捕获任何异常的catch块

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    double m, n;
```

```
    cin >> m >> n;
```

```
    try {
```

```
        cout << "before dividing." << endl;
```

```
        if( n == 0 )
```

```
            throw -1; //抛出整型异常
```

```
        elseif( m == 0 )
```

```
            throw -1.0; //抛出double型异常
```

```
        else
```

```
            cout << m / n << endl;
```

```
        cout << "after dividing." << endl;
```

```
    }
```

```

    catch(double d) {
        cout << "catch(double) " << d << endl;
    }
    catch(...) {    //匹配任何类型的异常
        cout << "catch(...)" << endl;
    }
    cout << "finished" << endl;
    return 0;
}

```

程序运行结果:

9 0 ☒

before dividing.
catch(...)
finished

0 6 ☒

before dividing.
catch(double) -1
finished

如果一个函数在执行的过程中,

- 抛出的异常在本函数内就被catch块捕获并处理了, 那么该异常就不会抛给这个函数的调用者 (也称“上一层的函数”)

- 如果异常在本函数中没被处理, 就会被抛给上一层的函数

//protram 20.4.3.cpp 异常再抛出

```
#include <iostream>
#include <string>
using namespace std;
class CException{
    public :
        string msg;
        CException(string s):msg(s) { }
};
```



```
double Devide(double x, double y){  
    if( y == 0 )  
        throw CException("devided by zero");  
    cout << "in Devide" << endl;  
    return x / y;  
}
```

```
int CountTax( int salary ){  
    try {  
        if( salary < 0 )  
            throw -1;  
        cout << "counting tax" << endl;  
    }  
    catch ( int ) {  
        cout << "salary < 0" << endl;  
    }  
}
```

```
        cout << "tax counted" << endl;
        return salary * 0.15;
    }
int main(){
    double f= 1.2;
    try {
        CountTax(-1);
        f = Devide(3, 0);
        cout << "end of try block" << endl;
    }
    catch( CException e ) {
        cout << e.msg << endl;
    }
    cout << "f=" << f << endl;
    cout << "finished" << endl;
    return 0;
}
```

输出结果:

salary < 0

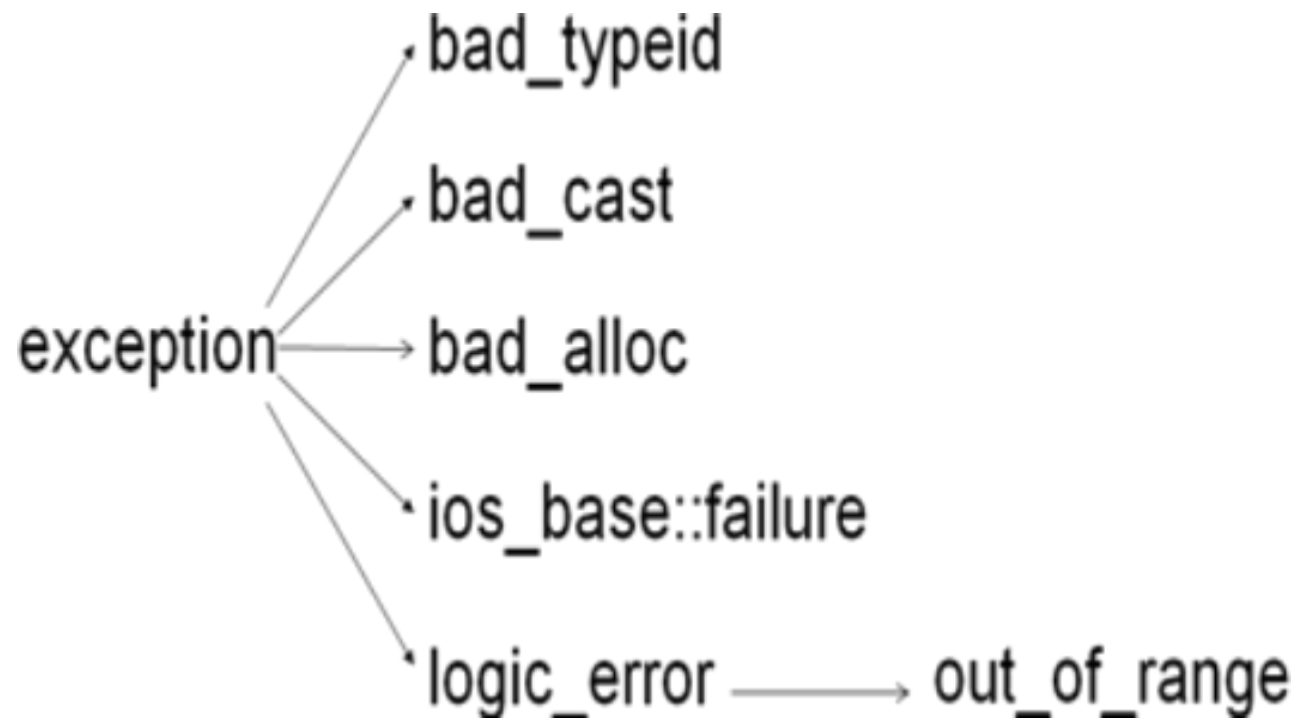
tax counted

devided by zero

f=1.2

finished

C++标准库中有一些类代表异常，这些类都是从 **exception** 类派生而来，常用的几个异常类如下：



bad_cast

在用 dynamic_cast进行从多态基类对象(或引用), 到派生类的引用的强制类型转换时, 如果转换是不安全的, 则会抛出此异常

//program 20.4.5.cpp bad_cast异常

```
#include <iostream>
#include <stdexcept>
using namespace std;
class Base {
    virtual void func(){}
};
class Derived : public Base {
public:
    void Print() { }
};
```

```
void PrintObj( Base & b){  
    try {  
        Derived & rd = dynamic_cast<Derived&>(b);  
        //此转换若不安全, 会抛出bad_cast异常  
        rd.Print();  
    }  
    catch (bad_cast & e) {  
        cerr << e.what() << endl;  
    }  
}  
  
int main (){  
    Base b;  
    PrintObj(b);  
    return 0;  
}
```

输出结果:
Bad dynamic_cast!

bad_alloc

在用new运算符进行动态内存分配时，如果没有足够的内存，则会引发此异常

//program 20.4.6.cpp bad_alloc异常

```
#include <iostream>
#include <stdexcept>
using namespace std;
int main (){
    try {
        char * p = new char[0x7fffffff]; //无法分配这么多空间, 会抛出异常
    }
    catch (bad_alloc & e) {
        cerr << e.what() << endl;
    }
    return 0;
}
```

输出结果:

bad allocation

out_of_range

用vector或string的at成员函数根据下标访问元素时, 如果下标越界, 就会抛出此异常。例如:

//program 20.4.7.cpp out_of_range异常

```
#include <iostream>
#include <stdexcept>
#include <vector>
#include <string>
using namespace std;
int main (){
    vector<int> v(10);
        try {
            v.at(100)=100; //抛出out_of_range异常
        }
        catch (out_of_range & e) {
            cerr << e.what() << endl;
```



```
string s = "hello";  
try {  
    char c = s.at(100); //抛出out_of_range异常  
}  
catch (out_of_range & e) {  
    cerr << e.what() << endl;  
}  
return 0;  
}
```

输出结果:

invalid vector<T> subscript
invalid string position

 C++语言中，**预编译**（编译预处理）指在真正开始对程序编译前，对源程序进行一番处理，形成一个临时源程序文件

 预编译不会修改程序

 主要包含:

- 符号常量的定义和宏定义 (内联函数)
- 文件包含
- 条件编译

```
#define DEBUG_VERSION
```

```
#ifdef DEBUG_VERSION
```

第一部分

```
#else
```

第二部分

```
#endif
```

- 如果有 `#define DEBUG_VERSION` 则第一部分被编译，
否则第二部分被编译
- **`#ifndef`** 则相反

□ 因此编写.h文件时, 往往这样写:

```
#ifndef SOMEHEAD_H
```

```
#define SOMEHEAD_H
```

头文件具体内容

```
#endif
```

□ 这样能避免在多个文件的工程中, 有的头文件可能被重复包含的问题

1) 如何使用在其他文件中定义的全局变量?

a.cpp 里定义了全局变量 `int a;`

b.cpp 里要用, 则如下声明:

`extern int a;`

2) 如何使用在其他文件中定义的全局函数?

a.cpp里写了函数 `void fun() { }`

b.cpp里要用, 则在b.cpp里写出函数声明即可:

`extern void fun();`

3) 如何使用在其他文件中写的类?

a.h 写类的声明, a.cpp里写类的成员函数的实现, 那么在 b.cpp里只要**`#include <a.h>`**就可以使用该类了

4) 如何写一个类库或函数库给别人用, 却不让别人看到源代码?

- 把你的函数或类实现在 `myclass.cpp` 里
- 函数或类的声明放在 `myclass.h` 里
- 编译后, 得到 `myclass.obj` 文件

把 `myclass.h` 和 `myclass.obj` 提供给别人,
别人 `#include <myclass.h>`

然后在编译器的链接选项里指定要链接 `myclass.obj` 即可

5) 如果有一个别人用C写的函数库我要在C++工程里用, 怎么办?

在别人提供的函数声明前加 extern "C"

```
extern "C" void OthersFunction();
```

否则会在链接的时候找不到函数

```
extern "C" {  
    void OthersFunctoion();  
    .....  
}
```

或者将所有别人的C函数的声明都写在上面的花括号中间

Thanks !

